

A polynomial-time algorithm for recognizing high-bandwidth graphs

Luis M. B. Varona^{1,2,3}

January 27, 2026

Abstract

An unweighted, undirected graph is said to have *bandwidth* at most k if its nodes can be labelled with distinct integers such that no two adjacent nodes have labels that differ by more than k . The paper [Journal of Algorithms **5** (1984), no. 4, 531–546] presented a dynamic programming algorithm to determine whether a graph on n nodes has bandwidth $\leq k$ in $O(n^k)$ time and $O(n^k)$ space; for small fixed k close to 0, this is effectively polynomial, but as k scales with n , it becomes exponential. In this paper, we reformulate the question of whether a valid labelling exists as a bipartite matching problem for sufficiently large $k \geq \lfloor (n-1)/2 \rfloor$. This allows us to use Hall’s marriage theorem to develop an algorithm that runs in $O(n^{n-k+1})$ time and $O(n^2)$ space, yielding polynomial complexity for large k close to n . Combined with the aforementioned 1984 algorithm (which is efficient for small k), this demonstrates that the bandwidth recognition problem is solvable in polynomial time whenever either k or $n - k$ remains small.

Keywords: graph algorithms, graph bandwidth, bandwidth reduction, Hall’s marriage theorem

ACM Classification: G.2.2; F.2.2

1 Introduction

Let G be an unweighted, undirected finite graph on n nodes with no self-loops. The *bandwidth* of G under some labelling (or “layout”) π of its nodes is the minimum non-negative integer $k \in \{0, 1, \dots, n-1\}$ such that $|\pi(u) - \pi(v)| \leq k$ for every edge $\{u, v\}$ in G ; we denote this quantity by $\beta_\pi(G)$. A long-standing problem in graph theory and combinatorics is to find a layout π for which $\beta_\pi(G)$ is small. Many computational processes involve sparse symmetric matrices that can be viewed as adjacency matrices, and reducing the bandwidth of the corresponding graphs helps improve properties like cache locality and low storage space. As such, graph bandwidth reduction has many practical applications in engineering and scientific computing, including solving linear systems, approximating partial differential equations, optimizing circuit layout, and more [MS14].

There are two variants of the bandwidth reduction problem: *minimization* (finding a layout π that minimizes or nearly minimizes $\beta_\pi(G)$) and *recognition* (determining whether there exists a layout π such that $\beta_\pi(G) \leq k$ for some fixed integer $k \geq 0$). Although most bandwidth reduction applications derive from minimization algorithms (see, e.g., those described in [MS14]), bandwidth recognition (the primary subject of this paper) also has applications in other fields of theoretical mathematics and computer science. In computational complexity theory, for instance, [RSS3] showed that a graph on n nodes with bandwidth $\leq k$ requires at most $\min\{2k^2 + k + 1, 2k \log_2 n\}$ pebbles in the computational pebble game. Quantum information theorists, too, are interested in determining whether the density matrix of a quantum state (or rather, the associated graph) has bandwidth $\leq k$, since this is necessary (although not sufficient) for a property called “ k -incoherence” [JMP25].

¹Department of Politics & International Relations, Mount Allison University, Sackville, NB, Canada E4L 1E4

²Department of Economics, Mount Allison University, Sackville, NB, Canada E4L 1E4

³Department of Mathematics & Computer Science, Mount Allison University, Sackville, NB, Canada E4L 1E4

The foundational work of [Sax80] provided the first general-case algorithm to determine whether a graph on n nodes has bandwidth at most k , requiring $O(n^{k+1})$ time and space. This algorithm was later improved upon by [GS84], who reduced both time and space complexity to $O(n^k)$. The most recent work in the field was conducted by [DCM99] and [CSG05]—although the main contribution of each study was an (exact) bandwidth minimization algorithm, both algorithms iteratively invoke their own recognition algorithms as subroutines. All existing approaches, however, are optimized for low-bandwidth recognition, exhibiting polynomial complexity only for small k —when k scales with n , these algorithms become exponential.

The present work addresses this gap in the literature, being the first to tackle the problem of high-bandwidth recognition (for k close to the maximum possible bandwidth of $n - 1$). In particular, we present an algorithm that requires $O(n^{n-k+1})$ time and $O(n^2)$ auxiliary space, making it effectively polynomial for large k as opposed to small k . Our algorithm begins by iterating over all possible assignments of the first $n - k - 1$ nodes. We then exploit Hall’s marriage theorem upon each iteration to efficiently verify whether the remaining nodes can be placed to achieve bandwidth $\leq k$, breaking early from the loop if a valid layout is successfully found. This represents the first work to explicitly address the bandwidth recognition problem since 1984 [GS84], and the first to advance recognition algorithms in any capacity (indirect or otherwise) since 2005 [CSG05].

1.1 Organization of the paper

We begin in Section 2 by introducing the notation used throughout the paper and covering preliminary definitions and results. In Section 3, we build on these concepts, using some implications of Hall’s marriage theorem to lay the theoretical groundwork for the algorithm. We then provide a detailed description of our algorithm in Section 4, accompanied by a formal complexity analysis and comparisons to prior literature on bandwidth recognition. In Section 5, we benchmark our algorithm against these previous approaches, demonstrating that it is generally more efficient for large values of k close to the maximum bandwidth of $n - 1$. Finally, we close in Section 6 with some open questions and suggestions for future work.

2 Preliminaries and definitions

We exclusively consider unweighted, undirected, finite graphs with no self-loops. Given a graph G , we write $V(G)$ for the node set of G , $E(G)$ for the edge set of G , and $A(G)$ for the adjacency matrix of G . Although less frequently, we also use $\varepsilon_G(v)$ to denote the eccentricity of v in G (i.e., the maximum shortest-path distance between v and any other node in G) and $N_{G,k}(v)$ to denote the k -hop neighbourhood of v in G (i.e., the set of nodes in G at distance between 1 and k from v).

We follow the standard convention of 0-based indexing for all data structures and algorithms. Given an array `arr` and a hash map `map`, we write `arr[i]` for the element of `arr` at index i and `map[k]` for the value in `map` associated with the key k . Moreover, we assume constant-time arithmetic and array access in all our complexity evaluations (rather than measure complexity in terms of bit operations).

Finally, on the topic of function notation: we use $\text{im}(f)$ to denote the image of the map f , $g : A \hookrightarrow B$ to indicate that $g : A \rightarrow B$ is bijective, and $h : C \hookrightarrow D$ to indicate that $h : C \rightarrow D$ is injective.

We now introduce further terminology—as well as formalize some existing terms already introduced in Section 1—necessary to understand our algorithm.

2.1 Terminology for the algorithm

Definition 1 (Layout). *Let G be a graph on n nodes. A layout π of G is a bijective map $\pi : V(G) \hookrightarrow \{0, 1, \dots, n - 1\}$, labelling the nodes of G with distinct consecutive non-negative integers starting from 0.*

Definition 2 (Bandwidth). *Let G be a graph on n nodes and π be a layout of G . The layout bandwidth of G with respect to π , denoted by $\beta_\pi(G)$, is the minimum non-negative integer $k \geq 0$ such that $|\pi(u) - \pi(v)| \leq k$ for all edges $\{u, v\} \in E(G)$. The graph bandwidth (or simply bandwidth) of G , denoted by $\beta(G)$, is the minimum value of $\beta_\varphi(G)$ over all layouts φ of G .*

Before moving on, we note some basic facts regarding graph bandwidth that will later prove useful.

Fact 1. Given a graph G on n nodes, $\beta(G)$ is always at most $n-1$, since the maximum difference in labels of any two nodes $u, v \in V(G)$ is achieved when $\pi(u) = 0$ and $\pi(v) = n-1$.

Fact 2. For any graph G , $\beta(G)$ is equal to the maximum bandwidth of its connected components. This follows immediately from the observation that connected components share no edges and can therefore be arranged sequentially in any layout. The overall bandwidth is thus determined solely by the component with the largest bandwidth.

As previously noted, the focus of this paper is developing an efficient algorithm to determine whether $\beta(G) \leq k$ for some graph G and integer k . Of course, we are primarily concerned with *high-bandwidth* graph recognition in particular—that is, determining whether $\beta(G) \leq k$ for “large” k . Specifically, we require $\lfloor \frac{n-1}{2} \rfloor \leq k < n-1$, so that mappings which we call left and right partial layouts—defined below—have nonempty domains that do not overlap with each other.

Definition 3 (Partial layouts). Let G be a graph on n nodes and $k \in \{\lfloor \frac{n-1}{2} \rfloor, \lfloor \frac{n-1}{2} \rfloor + 1, \dots, n-2\}$ be an integer. Then:

- (i) A left partial layout of G (with respect to bandwidth k) is an injective map $\mathcal{L} : \{0, 1, \dots, n-k-2\} \hookrightarrow V(G)$, determining the first $n-k-1$ nodes placed by some layout of G .
- (ii) A right partial layout of G (again w.r.t. bandwidth k) is an injective map $\mathcal{R} : \{k+1, k+2, \dots, n-1\} \hookrightarrow V(G)$, determining the last $n-k-1$ nodes placed by some layout of G .
- (iii) A left partial layout $\mathcal{L}$ and a right partial layout $\mathcal{R}$ (both w.r.t. bandwidth k) are said to be compatible if there exists some layout π of G such that π^{-1} agrees with both $\mathcal{L}$ and $\mathcal{R}$ on their respective domains (or equivalently, if $\text{im}(\mathcal{L}) \cap \text{im}(\mathcal{R}) = \emptyset$). Moreover, we refer to $\mathcal{L}$ and $\mathcal{R}$ as the left and right partial layouts induced by π (w.r.t. bandwidth k).

The domain of any left or right partial layout (with respect to bandwidth k) consists of precisely $n-k-1$ positions, so it is now clear why we restrict our attention to the case that $\lfloor \frac{n-1}{2} \rfloor \leq k < n-1$. If $k = n-1$, then $n-k-1 = 0$, so any left or right partial layout (w.r.t. bandwidth k) will have an empty domain. (Indeed, this is a trivial case—as noted in Fact 1, $\beta(G) \leq n-1$ for every graph G on n nodes, and any layout suffices.) If $k < \lfloor \frac{n-1}{2} \rfloor$, then $2(n-k-1) > n$, indicating that the domains of any left and any right partial layout (w.r.t. bandwidth k) will not be disjoint.

This would defeat the entire purpose of separately specifying which nodes occupy the leftmost and rightmost positions of a layout to reduce the search space for bandwidth recognition. As will soon be formalized in Lemma 3.1, whether a layout π of G satisfies $\beta_\pi(G) \leq k$ depends solely on which nodes occupy these outer positions—namely, $\{0, 1, \dots, n-k-2\}$ on the left and $\{k+1, k+2, \dots, n-1\}$ on the right—since only edges between these two regions can have endpoints whose labels differ by more than k . Keeping the domains disjoint allows us to treat these two regions as independent, so that we may enumerate all possible left partial layouts and, for each, determine whether a valid right partial layout exists that induces bandwidth $\leq k$. (Theorem 3.2 provides an efficient criterion for this.)

We are now prepared to present some preliminary mathematical results that will lay the foundation for our algorithm in Section 4.

3 Preliminary results

We first describe when a layout π of a graph G satisfies $\beta_\pi(G) \leq k$, in terms of edges bridging left and right partial layouts.

Lemma 3.1. Let G be a graph on n nodes and $k \in \{\lfloor \frac{n-1}{2} \rfloor, \lfloor \frac{n-1}{2} \rfloor + 1, \dots, n-2\}$ be an integer. A layout π of G satisfies $\beta_\pi(G) \leq k$ if and only if $\{\pi^{-1}(i), \pi^{-1}(k+j+1)\} \notin E(G)$ for all $0 \leq i < n-k-1$ and $i \leq j < n-k-1$.

Proof. Let G , k , and π be as defined above, and suppose that $\beta_\pi(G) \leq k$. Then by definition, no edge has endpoints whose images under π differ by more than k . For any fixed i and j with $0 \leq i < n-k-1$

and $i \leq j < n - k - 1$, the positions i and $k + j + 1$ differ by $(k + j + 1) - i \geq k + 1 > k$, so certainly $\{\pi^{-1}(i), \pi^{-1}(k + j + 1)\} \notin E(G)$, and we have proven the forward direction of the lemma.

We now prove the backward direction by contrapositive: suppose that $\beta_\pi(G) > k$. Then there exists an edge $\{u, v\} \in E(G)$ with $|\pi(u) - \pi(v)| > k$. Without loss of generality, assume that $\pi(u) < \pi(v)$, so $\pi(v) - \pi(u) \geq k + 1$. Observe that we can bound $\pi(u)$ from below and above by $\pi(u) \geq 0$ and

$$\pi(u) \leq \pi(v) - (k + 1) \leq (n - 1) - (k + 1) < n - k - 1;$$

similarly, we have $\pi(v) \geq \pi(u) + k + 1$ and $\pi(v) \leq n - 1$. Finally, setting $i = \pi(u)$ and $j = \pi(v) - (k + 1)$ (so that $0 \leq i < n - k - 1$ and $i \leq j < n - k - 1$), we see that

$$\{u, v\} = \{\pi^{-1}(\pi(u)), \pi^{-1}(\pi(v))\} = \{\pi^{-1}(i), \pi^{-1}(k + j + 1)\}$$

forms an edge. By contrapositive, it follows that $\beta_\pi(G) \leq k$ if no such edge exists, thus completing the proof. $\square$

It then follows that whether a layout π of a graph G on n nodes satisfies $\beta_\pi(G) \leq k$ depends solely on the left and right partial layouts it induces (at least when $k \geq \lfloor \frac{n-1}{2} \rfloor$), motivating the following definition.

Definition 4 (Feasible pair). *Let G be a graph on n nodes, $k \in \{\lfloor \frac{n-1}{2} \rfloor, \lfloor \frac{n-1}{2} \rfloor + 1, \dots, n - 2\}$ be an integer, and $\mathcal{L}$ and $\mathcal{R}$ be compatible left and right partial layouts of G (with respect to bandwidth k). The pair $(\mathcal{L}, \mathcal{R})$ is called feasible (again w.r.t. bandwidth k) if $\beta_\pi(G) \leq k$ for any layout π that induces $\mathcal{L}$ and $\mathcal{R}$ as its left and right partial layouts.*

(Equivalently, per Lemma 3.1: $(\mathcal{L}, \mathcal{R})$ is feasible w.r.t. bandwidth k if and only if $\{\mathcal{L}(i), \mathcal{R}(k + j + 1)\} \notin E(G)$ for all $0 \leq i < n - k - 1$ and $i \leq j < n - k - 1$.) Using only Lemma 3.1, it is already possible to formulate a naïve algorithm to check whether $\beta(G) \leq k$. Although we refrain from a comprehensive analysis for the sake of brevity, we briefly describe this algorithm below.

1. Enumerate all left partial layouts of G . There are exactly

$$\frac{n!}{(n - (n - k - 1))!} \in O(n^{n-k-1})$$

of these, since we are taking all $(n - k - 1)$ -permutations without repetition of all n nodes.

2. For each left partial layout $\mathcal{L}$, enumerate all right partial layouts of G compatible with $\mathcal{L}$. There are, again,

$$\frac{(k + 1)!}{((k + 1) - (n - k - 1))!} \in O((k + 1)^{n-k-1}) \subseteq O(n^{n-k-1})$$

such right partial layouts, since we are taking all $(n - k - 1)$ -permutations without repetition of the remaining $n - (n - k - 1) = k + 1$ nodes.

3. For each of the resulting $(\mathcal{L}, \mathcal{R})$ pairs, check whether $(\mathcal{L}, \mathcal{R})$ is feasible, and return early if so. As per Lemma 3.1, we must verify $\{\mathcal{L}(i), \mathcal{R}(k + j + 1)\} \notin E(G)$ for all $0 \leq i < n - k - 1$ and $i \leq j < n - k - 1$ for each pair, amounting to

$$\sum_{i=0}^{n-k-2} (n - k - i - 1) = \frac{(n - k - 1)(n - k)}{2} \in O((n - k)^2) \subseteq O(n^2)$$

operations total.

It is clear that this procedure requires $O(n^{n-k-1} \cdot n^{n-k-1} \cdot n^2) = O(n^{2(n-k)})$ time. While this is already effectively polynomial for large k close to the maximum possible bandwidth of $n - 1$, our results below allow for an even more efficient approach. Rather than enumerating all $O(n^{n-k-1})$ compatible right partial layouts for each left partial layout, we can determine whether a valid (i.e., inducing bandwidth $\leq k$) one exists far more efficiently using Hall's marriage theorem. As we will cover in our complexity analysis in Subsection 4.1, this reduces the overall time complexity to $O(n^{n-k+1})$ —a substantial improvement over the naïve $O(n^{2(n-k)})$ approach described above.

Theorem 3.2. Let G be a graph on n nodes, $k \in \{\lfloor \frac{n-1}{2} \rfloor, \lfloor \frac{n-1}{2} \rfloor + 1, \dots, n-2\}$ be an integer, and $\mathcal{L}$ be a left partial layout of G (with respect to bandwidth k). Additionally, define the sets

$$A_j := \{v \in V(G) \setminus \text{im}(\mathcal{L}) : \{\mathcal{L}(i), v\} \notin E(G) \text{ for all } i \leq j\}$$

for all $0 \leq j < n - k - 1$. Then there exists a right partial layout $\mathcal{R}$ (w.r.t. bandwidth k) compatible with $\mathcal{L}$ such that $(\mathcal{L}, \mathcal{R})$ is feasible if and only if

$$|A_j| \geq n - k - j - 1 \quad \text{for all } 0 \leq j < n - k - 1.$$

Proof. Let G , k , $\mathcal{L}$, and the A_j 's be as defined above. We now consider whether there exists a compatible right partial layout $\mathcal{R}$ such that $(\mathcal{L}, \mathcal{R})$ is feasible. By Lemma 3.1/Definition 4, $(\mathcal{L}, \mathcal{R})$ is feasible if and only if $\{\mathcal{L}(i), \mathcal{R}(k+j+1)\} \notin E(G)$ for all $0 \leq i < n - k - 1$ and $i \leq j < n - k - 1$. By construction of the A_j 's, this is equivalent to requiring $\mathcal{R}(k+j+1) \in A_j$ for all $0 \leq j < n - k - 1$.

As such, we can reformulate this as a bipartite matching problem, where $\text{domain}(\mathcal{R}) = \{k+1, k+2, \dots, n-1\}$ is on one side, $V(G) \setminus \text{im}(\mathcal{L})$ is on the other, and each $k+j+1 \in \text{domain}(\mathcal{R})$ is adjacent to all $v \in A_j \subseteq V(G) \setminus \text{im}(\mathcal{L})$. Hall's marriage theorem then tells us that we can construct such a matching if and only if for all subsets $S \subseteq \{0, 1, \dots, n - k - 2\}$,

$$\left| \bigcup_{j \in S} A_j \right| \geq |S|.$$

It is useful to observe here that the A_j 's are nested, with $A_0 \supseteq A_1 \supseteq \dots \supseteq A_{n-k-2}$. Therefore, we have

$$\bigcup_{j \in S} A_j = A_{\min S},$$

and so we see that Hall's condition for a feasible $(\mathcal{L}, \mathcal{R})$ pair is equivalent to having $|A_{\min S}| \geq |S|$ for all subsets S .

For any fixed j , the most restrictive constraint arises when $\min S = j$ and $|S|$ is maximized—namely, when $S = \{j, j+1, \dots, n - k - 2\}$. This gives $|S| = n - k - j - 1$, so indeed $|A_{\min S}| \geq |S|$ only if $|A_j| \geq n - k - j - 1$ for all $0 \leq j < n - k - 1$. Conversely, if $|A_j| \geq n - k - j - 1$ for all $0 \leq j < n - k - 1$, then for any subset S with $\min S = j$, we have

$$|A_{\min S}| = |A_j| \geq n - k - j - 1 = |\{j, j+1, \dots, n - k - 2\}| \geq |S|,$$

so $|A_{\min S}| \geq |S|$ if $|A_j| \geq n - k - j - 1$ for all $0 \leq j < n - k - 1$.

We have therefore shown that there exists a right partial layout $\mathcal{R}$ such that $(\mathcal{L}, \mathcal{R})$ is feasible if and only if $|A_j| \geq n - k - j - 1$ for all $0 \leq j < n - k - 1$, exactly as desired. $\square$

In addition to providing a condition for when it is possible to find a feasible $(\mathcal{L}, \mathcal{R})$ pair for a given left partial layout $\mathcal{L}$, these results also allow us to explicitly construct such an $\mathcal{R}$; we present this procedure below as a corollary.

Corollary 3.3. Let G , k , $\mathcal{L}$, and the A_j 's be as defined in Theorem 3.2, and suppose that the Hall conditions $|A_j| \geq n - k - j - 1$ are satisfied for all $0 \leq j < n - k - 1$. Then we can construct a right partial layout $\mathcal{R}$ compatible with $\mathcal{L}$ such that $(\mathcal{L}, \mathcal{R})$ is feasible as follows: for each $j = n - k - 2, n - k - 3, \dots, 0$ (in decreasing order), define $\mathcal{R}(k+j+1)$ to be any node in A_j not already selected in a previous step.

Proof. It suffices to show that at each step j , there exists at least one element of A_j not yet selected to be in the image of $\mathcal{R}$. When processing some fixed index j (counting down from $n - k - 2$), we have already assigned exactly $n - k - j - 2$ nodes. By hypothesis, there exists a right partial layout $\mathcal{R}$ such that $(\mathcal{L}, \mathcal{R})$ is feasible; Theorem 3.2 tells us that $|A_j| \geq n - k - j - 1 = (n - k - j - 2) + 1$ is a necessary condition for this, so at least one element of A_j must still be available, and we are done. $\square$

Finally, before presenting the algorithm itself in Section 4, we collect some lower bounds on graph bandwidth that serve as early termination conditions for bandwidth recognition.

Proposition 3.4. *Let G be a graph with m edges and n nodes, and define the quantities $\alpha(G)$ and $\gamma(G)$ by*

$$\alpha(G) := \max_{v \in V(G)} \max_{1 \leq k \leq \varepsilon_G(v)} \left\lceil \frac{|N_{G,k}(v)|}{2k} \right\rceil \quad \text{and} \quad \gamma(G) := \min_{v \in V(G)} \max_{1 \leq k \leq \varepsilon_G(v)} \left\lceil \frac{|N_{G,k}(v)|}{k} \right\rceil.$$

Then:

- (i) $\alpha(G)$ and $\gamma(G)$ both bound $\beta(G)$ from below.
- (ii) $\alpha(G)$ and $\gamma(G)$ are both computable in $O(mn)$ time and $O(n^2)$ space.
- (iii) Neither of these bounds consistently dominates the other.

Proof. Propositions 4 and 6 of [CSG05] give exactly the statements of (i) and (ii); note that they include the node itself in their definition of a k -hop neighbourhood—for which they also use slightly different notation—so our $|N_{G,k}(v)|$ corresponds to their $|N_k(v)| - 1$. The paper also gives constructive procedures to compute both $\alpha(G)$ and $\gamma(G)$ in $O(mn)$ time (instead of merely proving that it is possible to do so in theory), allowing us to actually invoke these bounds in our algorithm later in Section 4. (Although space complexity claims are not explicitly presented, a closer look reveals that the dominating cost in any implementation is the storage of an $O(n^2)$ distance matrix.)

For (iii), [CSG05, p. 360] also note that there are graphs for which α is a tighter bound and others for which γ is a tighter bound; we illustrate this ourselves with some specific examples. For instance, although we omit the actual calculations here for brevity, it is straightforward to see that the complete multipartite graph $K_{n,n}$ gives

$$\alpha(K_{n,n}) = \left\lceil \frac{n}{2} \right\rceil < n = \gamma(K_{n,n})$$

for all $n \geq 2$, whereas the lollipop graph $L_{n,n}$ gives

$$\alpha(L_{n,n}) = \left\lceil \frac{n}{2} \right\rceil > 2 = \gamma(L_{n,n})$$

for all $n \geq 5$. As such, it is useful to apply both bounds to create early termination conditions for any bandwidth recognition algorithm, including ours. $\square$

Finally, we are in possession of all the mathematical preliminaries needed to develop our algorithm.

4 The algorithm

Once again, let G be a graph on n nodes and $k \in \{\lfloor \frac{n-1}{2} \rfloor, \lfloor \frac{n-1}{2} \rfloor + 1, \dots, n-2\}$ be an integer; we seek an answer to the question of whether $\beta(G) \leq k$. As we have seen, Theorem 3.2 allows us to check whether a given left partial layout $\mathcal{L}$ admits a compatible right partial layout $\mathcal{R}$ such that $(\mathcal{L}, \mathcal{R})$ is feasible—namely, by verifying that $|A_j| \geq n - k - j - 1$ for all $0 \leq j < n - k - 1$ (where the A_j ’s are as defined therein). However, computing each $|A_j|$ from scratch would require iterating over all $k+1$ unplaced nodes in $V(G) \setminus \text{im}(\mathcal{L})$ and checking adjacency to $\mathcal{L}(i)$ for all $0 \leq i \leq j$, amounting to

$$\sum_{j=0}^{n-k-2} (k+1)(j+1) = (k+1) \cdot \frac{(n-k-1)(n-k)}{2} \in O(k(n-k)^2)$$

operations. We therefore introduce two additional data structures—a hash map **blockedMap** and an array **blockedArr**—to help us verify this condition more efficiently (namely, in $O(nk)$ time per left partial layout).

Fix a left partial layout $\mathcal{L}$ and, for convenience, denote the set of unplaced nodes by $V' := V(G) \setminus \text{im}(\mathcal{L})$. For each unplaced node $v \in V'$, we define **blockedMap**[v] to be the minimum index i such that $\{\mathcal{L}(i), v\} \in E(G)$ (or ∞ by convention, if no $\mathcal{L}(i)$ is adjacent to v). Observe that $v \in A_j$ if and only if **blockedMap**[v] $> j$, since each $A_j \subseteq V'$ consists precisely of those nodes in V' that are not adjacent to any $\mathcal{L}(i)$ with $i \leq j$. We then define **blockedArr** to be an array containing the nodes in V' , sorted in increasing order by their

corresponding **blockedMap** values. This allows us to compute each $|A_j|$ in $O(\log k)$ time: since $|V'| = k + 1$ and a node $v \in V'$ belongs to A_j if and only if $\mathbf{blockedMap}[v] > j$, we have

$$|A_j| = (k + 1) - |\{v \in V' : \mathbf{blockedMap}[v] \leq j\}|.$$

The second term is precisely the number of nodes in **blockedArr** whose **blockedMap** values are at most j , which can be computed via binary search on the sorted array in $O(\log k)$ time.

If taken in isolation, we could then say that since we consider $j \in \{0, 1, \dots, n - k - 2\}$, it would take us $O((n - k) \log k)$ time to compute all the $|A_j|$'s and thus determine the existence of a feasible $(\mathcal{L}, \mathcal{R})$ pair for our fixed $\mathcal{L}$. However, we also need to take into consideration the costs of building **blockedMap** and sorting **blockedArr**. Building **blockedMap** takes $O(k(n - k))$ time, since for each of the $k + 1$ nodes in V' , we check adjacency to each of the $n - k - 1$ nodes in $\text{im}(\mathcal{L})$. Meanwhile, sorting **blockedArr** takes $O(k \log k)$ time, using any standard comparison-based algorithm.

Together with the cost of verifying all Hall conditions, the total time per left partial layout is then $O((n - k) \log k + k(n - k) + k \log k)$. Depending on the value of k relative to n , either the first or the second term can dominate; since they are all in $O(nk)$, we adopt this as our upper bound on time complexity per left partial layout. (We defer a formal verification of this bound, alongside explanations of why all three terms are important, to Subsection 4.1.)

Note that **blockedArr** also facilitates the construction of a right partial layout $\mathcal{R}$ such that $(\mathcal{L}, \mathcal{R})$ is feasible, as described in Corollary 3.3. Rather than explicitly compute all A_j 's (which would require unnecessary space allocation) and track which nodes have already been assigned, we can simply use the last $n - k - 1$ elements of **blockedArr** (i.e., those with the largest **blockedMap** values) to construct $\mathcal{R}$. We formalize this procedure below.

Proposition 4.1. *Let G , k , $\mathcal{L}$, and the A_j 's be as defined in Theorem 3.2, and suppose that the Hall conditions $|A_j| \geq n - k - j - 1$ are satisfied for all $0 \leq j < n - k - 1$. Then the right partial layout $\mathcal{R}$ defined by*

$$\mathcal{R}(k + j + 1) := \mathbf{blockedArr}[2k - n + j + 2]$$

for all $0 \leq j < n - k - 1$ forms a feasible pair with $\mathcal{L}$.

Proof. Corollary 3.3 guarantees that when constructing $\mathcal{R}$ by iterating over $j = n - k - 2, n - k - 3, \dots, 0$ (in decreasing order) and assigning any unused node from A_j to $\mathcal{R}(k + j + 1)$ at each step, at least one such node is always available. We show that $\mathbf{blockedArr}[2k - n + j + 2]$ is always such a node.

As in the statement of the proposition, define the right partial layout $\mathcal{R}$ by

$$\mathcal{R}(k + j + 1) := \mathbf{blockedArr}[2k - n + j + 2]$$

for all $0 \leq j < n - k - 1$, and recall that **blockedArr** is an array of the $k + 1$ nodes in $V' := V(G) \setminus \text{im}(\mathcal{L})$, sorted in increasing order by their corresponding **blockedMap** values. The last $n - k - 1$ elements of this array occupy indices $2k - n + 2$ through to k , so these are precisely the elements in the image of $\mathcal{R}$.

Theorem 3.2 tells us that it suffices to verify that $\mathcal{R}(k + j + 1) \in A_j$ for all $0 \leq j < n - k - 1$ to show that $(\mathcal{L}, \mathcal{R})$ is feasible. By construction, a node v belongs to A_j if and only if $\mathbf{blockedMap}[v] > j$. Since

$$|A_j| = (k + 1) - |\{v \in V' : \mathbf{blockedMap}[v] \leq j\}|,$$

the Hall condition $|A_j| \geq n - k - j - 1$ (which holds by our initial hypothesis) implies that

$$|\{u \in V' : \mathbf{blockedMap}[u] \leq j\}| \leq (k + 1) - (n - k - j - 1) = 2k - n + j + 2.$$

Therefore, nodes at indices $2k - n + j + 2$ and beyond in **blockedArr** have **blockedMap** values exceeding j , placing them in A_j . This directly implies that

$$\mathcal{R}(k + j + 1) = \mathbf{blockedArr}[2k - n + j + 2] \in A_j$$

for all $0 \leq j < n - k - 1$, so $(\mathcal{L}, \mathcal{R})$ is indeed feasible, as desired. $\square$

Finally, we remark that although the algorithm as presented technically applies to disconnected graphs as well, it is more efficient in practice to run it separately on each connected component, since the bandwidth of any graph is equal to the maximum bandwidth of its connected components (Fact 2). Having said that, the full procedure is given in the Algorithm 1 pseudocode.

Algorithm 1 Determine whether $\beta(G) \leq k$ and, if so, find a layout π of G such that $\beta_\pi(G) \leq k$

```

1: function HIGHBANDWIDTHRECOGNITION(graph  $G$ , integer  $k$ )    ▷ For  $\lfloor \frac{|V(G)|-1}{2} \rfloor \leq k < |V(G)| - 1$ 
2:   let  $n \leftarrow |V(G)|$                                        ▷ Simply for convenience of notation

3:   if  $k < \max\{\alpha(G), \gamma(G)\}$  then    ▷ By Proposition 3.4(iii), it is helpful to compute both bounds
4:     return (false, null)
5:   end if

   ▷ We use this array to implicitly represent a right partial layout (it is more efficient to preallocate
   it here outside of the loop, as its entries will be overwritten upon each iteration anyway)
6:   let right  $\leftarrow$  [an empty array of length  $n - k - 1$ ]

7:   for each left partial layout  $\mathcal{L} : \{0, 1, \dots, n - k - 2\} \hookrightarrow V(G)$  do
8:     let  $V' \leftarrow V(G) \setminus \text{im}(\mathcal{L})$ 
9:     let blockedMap  $\leftarrow$  [an empty hash map from nodes to integers]

    ▷  $v \in A_j$  if and only if blockedMap[ $v$ ]  $> j$ , allowing for efficient computation of the  $|A_j|$ 's
10:    for each  $v \in V'$  do
11:      let blockedMap[ $v$ ]  $\leftarrow \min\{\{\infty\} \cup \{i : \{\mathcal{L}(i), v\} \in E(G)\}\}$ 
12:    end for

13:    let blockedArr  $\leftarrow$  [an array of all  $v \in V'$  sorted by their blockedMap[ $v$ ] values]
14:    let  $j \leftarrow 0$ 
15:    let validRight  $\leftarrow$  true

16:    while  $j < n - k - 1$  and validRight = true do
    ▷ We can efficiently compute  $|\{v \in V' : \text{blockedMap}[v] \leq j\}|$  term via binary search on the
    sorted blockedArr array, treating it as  $|\{i : \text{blockedMap}[\text{blockedArr}[i]] \leq j\}|$  instead
17:    let  $|A_j| \leftarrow k + 1 - |\{v \in V' : \text{blockedMap}[v] \leq j\}|$ 

    ▷ By Theorem 3.2, a feasible  $(\mathcal{L}, \mathcal{R})$  pair exists if and only if this inequality holds for all  $j$ 
18:    if  $|A_j| \geq n - k - j - 1$  then
    ▷ By Corollary 3.3/Proposition 4.1, this is a valid way to construct a right partial layout
    that forms a feasible pair with  $\mathcal{L}$ 
19:    let right[ $j$ ]  $\leftarrow$  blockedArr[ $2k - n + j + 2$ ]
20:    let  $j \leftarrow j + 1$ 
21:    else
22:      let validRight  $\leftarrow$  false
23:    end if
24:  end while

25:  if validRight = true then
26:    let  $\mathcal{R} \leftarrow$  [the right partial layout mapping each  $k + j + 1 \mapsto \text{right}[j]$ ]
    ▷ We can assign the nodes not in  $\mathcal{L}$  or  $\mathcal{R}$  to the remaining indices in any arbitrary order
27:    return (true, [any layout of  $G$  inducing  $\mathcal{L}$  and  $\mathcal{R}$  as left and right partial layouts])
28:  end if
29: end for

30: return (false, null)
31: end function

```

4.1 Complexity analysis

We now analyze the time and space complexity of Algorithm 1, proving that it requires $O(n^{n-k+1})$ time and $O(n^2)$ space.

Given a graph G on n nodes and a bandwidth threshold k , the algorithm begins by computing the α and γ lower bounds on $\beta(G)$, which, by Proposition 3.4(ii), each require $O(mn)$ time. If either bound exceeds k , the algorithm terminates immediately; otherwise, the main loop iterates over all left partial layouts (with respect to bandwidth k) of G . As mentioned earlier in Section 3, this is precisely the set of all $(n - k - 1)$ -permutations without repetition of $V(G)$ (which has n elements), amounting to

$$\frac{n!}{(n - (n - k - 1))!} \in O(n^{n-k-1})$$

left partial layouts total. Standard techniques for enumerating partial permutations allow each successive partial layout to be generated in $O(n)$ amortized time—for instance, [Jim98] gives an efficient procedure that first generates all $(n - k - 1)$ -combinations of n elements, then generates all permutations of each combination. Below, we show that the processing cost associated with each left partial layout is $O(nk)$, dominating this $O(n)$ (amortized) enumeration cost.

For each left partial layout $\mathcal{L}$, the algorithm performs the following operations. First, building the **blockedMap** hash map (lines 10–12) requires checking, for each of the $k + 1$ unplaced nodes in V' , adjacency to each of the $n - k - 1$ nodes in $\text{im}(\mathcal{L})$; this requires $O((k + 1)(n - k - 1)) = O(k(n - k))$ time. Second, sorting the **blockedArr** array (line 13) using any standard comparison-based algorithm requires $O(k \log k)$ time. Third, verifying the Hall conditions from Theorem 3.2 (lines 16–24) requires computing $n - k - 1$ $|A_j|$'s via binary search on **blockedArr**, which has exactly $k + 1$ elements; this requires $O((n - k - 1) \log(k + 1)) = O((n - k) \log k)$ time. (We also construct the **right** array—representing a right partial layout—concurrently with this step, avoiding a second pass over the data.)

Combining these contributions, we see that the time required to process each left partial layout is $O(k(n - k) + k \log k + (n - k) \log k)$. Each of these three terms is bounded above by $O(nk)$, with the first dominating when k is closer to $\lfloor \frac{n-1}{2} \rfloor$ (so $n - k > \log k$) and the second term dominating when k is closer to $n - 1$ (so $\log k > n - k$). Multiplying this $O(nk)$ processing cost by the $O(n^{n-k-1})$ left partial layouts, the total time requirement for the main loop is therefore $O(n^{n-k-1} \cdot nk) = O(n^{n-k}k)$.

LV: TODO: Wrap up proof of time complexity.

LV: TODO: Space complexity. $O(n^2)$ requirement comes really only from bound computation (need distance matrix); beyond this, show $O(n)$ auxiliary space (beyond storage of the input graph).

LV: TODO: Briefly note, again, that our algorithm is significantly better than the naïve $O(n^{2(n-k)})$ approach. Note also independence of sparsity, so works equally well on sparse and dense representations. (Indeed, this is what we are primarily concerned with, since k is large here. . .

LV: TODO: After everything is done, confirm that notation for big- O is consistent between here and Section 3.

4.2 Comparison to previous work

Whereas [GS84]'s $O(n^k)$ algorithm is effectively polynomial for small k close to the minimum bandwidth of 0 but exponential when k scales with n , our $O(n^{n-k+1})$ algorithm is effectively polynomial for large k close to the maximum possible bandwidth of $n - 1$ instead. The papers [DCM99] and [CSG05], on the other hand, present bandwidth minimization algorithms containing implicit recognition subroutines. (This is never explicitly addressed by the authors, but for instance, lines 4–6 of the pseudocode in [DCM99, Table 1] constitute a recognition algorithm in their own right.) No complexity claims are made about either the minimization algorithms presented or the implicit recognition subroutines, but any thorough analysis—which we omit here for brevity—shows that they are not polynomial for any regime of k values. (They do, however, incorporate effective pruning techniques that yield good running times in practice.) Hence, our algorithm is the only known method that is polynomial in time complexity for large k .

One limitation of our approach is that we are restricted to checking bandwidth $\leq k$ for $k \geq \lfloor \frac{n-1}{2} \rfloor$, whereas other existing bandwidth recognition algorithms can check bandwidth $\leq k$ for all $k \geq 0$. In any case, this is not a practical concern. Just as [GS84]'s $O(n^k)$ algorithm is exponential in complexity for

large k and thus impractical for such cases (despite it being theoretically applicable), we would not want to run our $O(n^{n-k+1})$ algorithm for small $k < \lfloor \frac{n-1}{2} \rfloor$ even if we could.

5 Experimental results

LV: TODO: Use [Var25] to compare to all three other algorithms. (Preliminary results indicate $10\times$ – $100\times$ speedups in relevant regimes of high k , although of course this is specifically the regimes for which our algorithm is optimized.)

- For [GS84], only for small matrices since it won't run on larger ones.
- For [DCM99], only MB-ID since MB-PS requires nontrivially optimized depth parameter. Both are comparable to each other.
- For [CSG05], only left-to-right since both-ways is harder to implement, and they are again comparable (neither dominating). Hybrid is never better than both so can exclude using same reasoning as for both-ways.
- Vary over n , then for each n vary over three different k . Choose ψ parameter to be random between $k - 2$ and k . For each (n, k) pair, randomly choose p parameter between 0.3 and 0.6 (see again [DCM99]). Collect raw data on all the runs in parquet or CSV, I guess, and include that in companion repo.
- That's for the affirmative answers; for the negative ones, vary again over n and then over k (different two k this time; *maybe* the first two from last time) then choose ψ to be random between $k + 1$ and $k + 3$. Repeat over and over until lower bound is $\leq k$.
- Note how [Var25] computes the α and γ bounds in $O(n^3)$ time instead of $O(mn)$ (not taking advantage of sparsity), but the complexity for the rest of the algorithm(s) is the same. Note also that preexisting code uses α and γ instead of the worse degree-based bound, which indeed makes the comparison even fairer to [GS84] and [DCM99]. Another note regarding α and γ : computed simultaneously in [Var25] (allowing the reuse of k -hop neighbourhoods, etc.).

6 Conclusion and future work

LV: TODO: Summarize results, including empirical improvements.

LV: TODO: Reword. The exponent in the original $O(n^k)$ algorithm is the difference between the threshold k and the minimum possible bandwidth of 0; by analogy, an ideal counterpart for high-bandwidth recognition would have a time complexity of $O(n^{n-k-1})$, with the exponent now representing the difference between the maximum possible bandwidth of $n-1$ and the threshold k . Although our $O(n^{n-k+1})$ algorithm does not perfectly achieve this duality, it remains a substantial result demonstrating that high-bandwidth recognition is decidable in polynomial time as well. Furthermore, our $O(n)$ auxiliary space requirement is even better than $O(n^{n-k-1})$; thus, we have actually achieved an improvement over duality to low-bandwidth recognition in terms of space.

Acknowledgements

I would like to thank Nathaniel Johnston, Liam Keliher, and Laurie Ricker for valuable advice and guidance. Additional gratitude is owed to Alexander B. Matheson, Serena Davis, Luc Campbell, Marco Cognetta, and Ryan Tifenbach for further helpful conversations.

References

- [CSG05] Alberto Caprara and Juan-José Salazar-González. Laying out sparse graphs with provably minimum bandwidth. *INFORMS Journal on Computing*, 17(3):356–373, 2005. [2](#), [6](#), [9](#), [10](#)
- [DCM99] Gianna M. Del Corso and Giovanni Manzini. Finding exact solutions to the bandwidth minimization problem. *Computing*, 62:189–203, 1999. [2](#), [9](#), [10](#)
- [GS84] Eitan M. Gurari and Ival Hal Sudborough. Improved dynamic programming algorithms for bandwidth minimization and the MinCut Linear Arrangement problem. *Journal of Algorithms*, 5(4):531–546, 1984. [2](#), [9](#), [10](#)
- [Jim98] Daniel A. Jiménez. Lecture 25: Generating permutations and combinations, 1998. Lecture notes for CS 3343, University of Texas at Austin. <https://www.cs.utexas.edu/~djimenez/utsa/cs3343/lecture25.html>. [9](#)
- [JMP25] Nathaniel Johnston, Shirin Moein, and Sarah Plosker. The factor width rank of a matrix. *Linear Algebra and its Applications*, 716:32–59, 2025. [1](#)
- [MS14] Liviu Octavian Maftiu-Scai. The bandwidths of a matrix. A survey of algorithms. *Annals of West University of Timisoara - Mathematics and Computer Science*, 52(2):183–223, 2014. [1](#)
- [RS83] Arnold L. Rosenberg and Ivan Hal Sudborough. Bandwidth and pebbling. *Computing*, 31:115–139, 1983. [1](#)
- [Sax80] James B. Saxe. Dynamic-programming algorithms for recognizing small-bandwidth graphs in polynomial time. *SIAM Journal on Algebraic and Discrete Methods*, 1(4):363–369, 1980. [2](#)
- [Var25] Luis M. B. Varona. MatrixBandwidth.jl: Fast algorithms for matrix bandwidth minimization and recognition. *Journal of Open Source Software*, 10(116), 2025. Article 9136. [10](#)